

TokenX | Complete Commands Guide

Protocol v3 - Root UID 0 / System UID 1000 / default route - Termux and Android shell reference.

Verified TokenX rish switches: --root, --system, default invocation and -c. Other commands below are standard Android shell commands executed through TokenX. Permissions vary by build, Binder identity and SELinux.

1. Setup and command discovery

```
cd ~
export RISH_APPLICATION_ID=com.termux
ls -l ./rish ./rish_shizuku.dex
chmod 700 ./rish
chmod 400 ./rish_shizuku.dex
./rish --help
```

The help switch is a discovery attempt; its availability and output depend on the installed build.

2. Shell routing - verified

```
./rish --root          # interactive Root shell
./rish --system        # interactive System shell
./rish                # configured default shell
./rish --root -c 'id; echo ROOT_OK'
./rish --system -c 'id; echo SYSTEM_OK'
./rish -c 'id; echo DEFAULT_OK'
```

Previously observed: Root yields UID 0; System yields UID 1000 via an isolated worker outside system_server. The default yielded UID 0 in one configuration. Shell UID 2000 fallback is not a confirmed dedicated rish flag.

3. Identity, device and SELinux

```
./rish --root -c 'id; whoami; id -Z'
./rish --system -c 'id; whoami; id -Z'
./rish --system -c 'getprop ro.product.model'
./rish --system -c 'getprop ro.build.version.release'
./rish --system -c 'getenforce; uptime'
```

4. Settings - read, enumerate, temporary write

```
./rish --system -c 'settings list system'
./rish --system -c 'settings list secure'
./rish --system -c 'settings list global'
./rish --system -c 'settings get system screen_off_timeout'
./rish --system -c 'settings get global device_name'
./rish --system -c '
settings put system tokenx_probe test
echo WRITE_EXIT=$?
settings get system tokenx_probe
settings delete system tokenx_probe
echo DELETE_EXIT=$?
'
```

A successful exit does not prove persistent Settings access. Confirm the written value and clean up the temporary key. UID 1000 may still encounter Binder or package permission checks.

5. AppOps - inspection

```
./rish --system -c 'cmd appops get com.termux'
./rish --system -c 'cmd appops get com.termux RUN_IN_BACKGROUND'
./rish --root -c 'cmd appops get com.termux RUN_IN_BACKGROUND'
./rish --system -c 'cmd appops get moe.shizuku.privileged.api'
./rish --system -c 'cmd appops help'
```

AppOps writes can alter app behavior. Capture the original mode and restore it after any controlled test.

6. Package manager and app details

```
./rish --system -c 'cmd package list packages'
./rish --system -c 'cmd package list packages -s'
./rish --system -c 'cmd package list packages --uid 1000'
./rish --system -c 'cmd package list packages -U | grep com.termux'
./rish --system -c 'pm path moe.shizuku.privileged.api'
./rish --system -c 'dumpsys package com.termux'
./rish --system -c 'dumpsys package moe.shizuku.privileged.api'
```

7. Services, Binder and system inspection

```
./rish --system -c 'service list'
./rish --system -c 'dumpsys power | head -60'
./rish --system -c 'dumpsys activity processes | head -60'
./rish --root -c 'ps -AZ | grep -Ei "tokenx|shizuku|system_server"'
./rish --root -c 'getprop | grep -Ei "tokenx|shizuku"'
```

8. Logs and troubleshooting

```
./rish --root -c 'logcat -d -t 500 | grep -Ei "tokenx|shizuku|binder"'
./rish --root -c 'logcat -d | grep "avc: denied" | tail -40'
./rish --root -c 'dmesg | tail -50'
./rish --system -c 'id; uptime'
```

UID -1: expected backend not active/verified. Timeout: possible manager identity, startup or transport issue.
Settings/AppOps denial: Binder identity or SELinux may apply even at UID 1000.

9. Manager identification and direct transport

```
export RISH_APPLICATION_ID=com.termux
export MANAGER_APPLICATION_ID=moe.shizuku.privileged.api
./rish --root -c 'id'
./rish --system -c 'id'
```

```
# Optional legacy direct test; only if DEX and class exist:
CLASSPATH="$HOME/tokenx_client.dex" /system/bin/app_process \
/system/bin rikka.shizuku.shell.TokenXTransportClient
```

The direct transport client previously returned HELLO, OPEN and PING with backendUid=1000. This optional test is not required for normal rish use. Manager package must match the installed manager.

10. Quick reference: Android commands inside -c

Command	Purpose
id / whoami / id -Z	Identity and SELinux context
getprop / setprop	Read / modify writable properties
settings get / list / put / delete	Android settings
cmd package / pm	Package information and actions
cmd appops	App operation state
cmd activity / am	Activity manager
cmd notification	Notification service
cmd deviceidle	Idle-state service
cmd power / cmd battery	Power and battery services
cmd user	User management
dumpsys	Service diagnostics
service list	Binder services
logcat / dmesg	Android and kernel logs
ps -AZ	Processes and security contexts
input keyevent	Simulated key input
wm size / wm density	Display configuration

Safety: Avoid system-server restarts, disabling SELinux, unreviewed AppOps changes or changing security settings just to obtain a passing test. Validate Root and System independently.

Build note: Based on observed Protocol v3 behavior and standard Android shell interfaces; not an exhaustive audit of the latest TokenX source.

11. One-command full diagnostic report

Run this in Termux. It saves a read-only Root/System diagnostic report to Downloads.

```
cd ~
export RISH_APPLICATION_ID=com.termux
OUT=/storage/emulated/0/Download/TokenX_Full_Diagnostics.txt
{
  echo '=== TOKENX DIAGNOSTICS ==='
  date
  for ROUTE in root system; do
    echo "=== $ROUTE ==="
    ./rish --$ROUTE -c '
      echo "[IDENTITY]"
      id; id -Z
      echo "[DEVICE]"
      getprop ro.product.model
      echo "[SETTINGS READ]"
      settings get system screen_off_timeout
      echo "[APPPOPS READ]"
      cmd appops get com.termux RUN_IN_BACKGROUND
      echo "[PACKAGES]"
      cmd package list packages --uid $(id -u) | head -15
      echo "[PROCESSES]"
      ps -A | grep -Ei "tokenx|shizuku|system_server" | head -15
    '
  done
} 2>&1 | tee "$OUT"
echo "Saved: $OUT"
```

Output: /storage/emulated/0/Download/TokenX_Full_Diagnostics.txt

If storage access is denied, grant Termux shared-storage access using `termux-setup-storage` and retry.